You can specify :aggregate_failure in an it method or a describe method. If you use aggregate_failure in a describe block, it applies to every spec inside the block. You can even specify aggregate_failure in the configuration to apply to all specs. That can cause problems, though, especially in integration tests where early failures often mean late code can't pass or might even be problematic to run.

No matter how you handle your specs, expectations that genuinely cover different branches of the application logic should be handled in separate specs. Avoid changing local variables inside a spec just to test different logic in the same test. That means you would want to keep these specs separate.

Prescription 15 Expectations that cover different branches of the application logic should be handled in separate specs.

```
it "knows full names" do
  user = User.create(:first_name => "Fred", :last_name => "Flintstone")
  expect(user.full_name).to eq("Fred Flintstone")
end

it "knows full names with a middle initial" do
  user = User.create(:first_name => "Fred", :last_name => "Flintstone"
      :middle_initial => "D")
  expect(user.full_name).to eq("Fred D. Flintstone")
end

it "knows full name where there's no first name" do
  user = User.create(:last_name => "Flintstone")
  expect(user.full_name).to eq("Flintstone")
end
```

Continually changing the value of user to put all those assertions in the same branch makes for a test that is very hard to read and understand.

Testing What Rails Gives You

Rails provides built-in functionality for associations and validations, which leads to questions about how to effectively and usefully test those features in your application.

The answer in both cases is similar, and goes back to the basic principle that we're testing functionality and not implementation. Although I do not normally write tests just to show the existence of a particular association or validation, I do sometimes write tests that show those features in action. For associations, this means showing the association in use. For validations, it means testing the overall logic of what makes an instance valid.